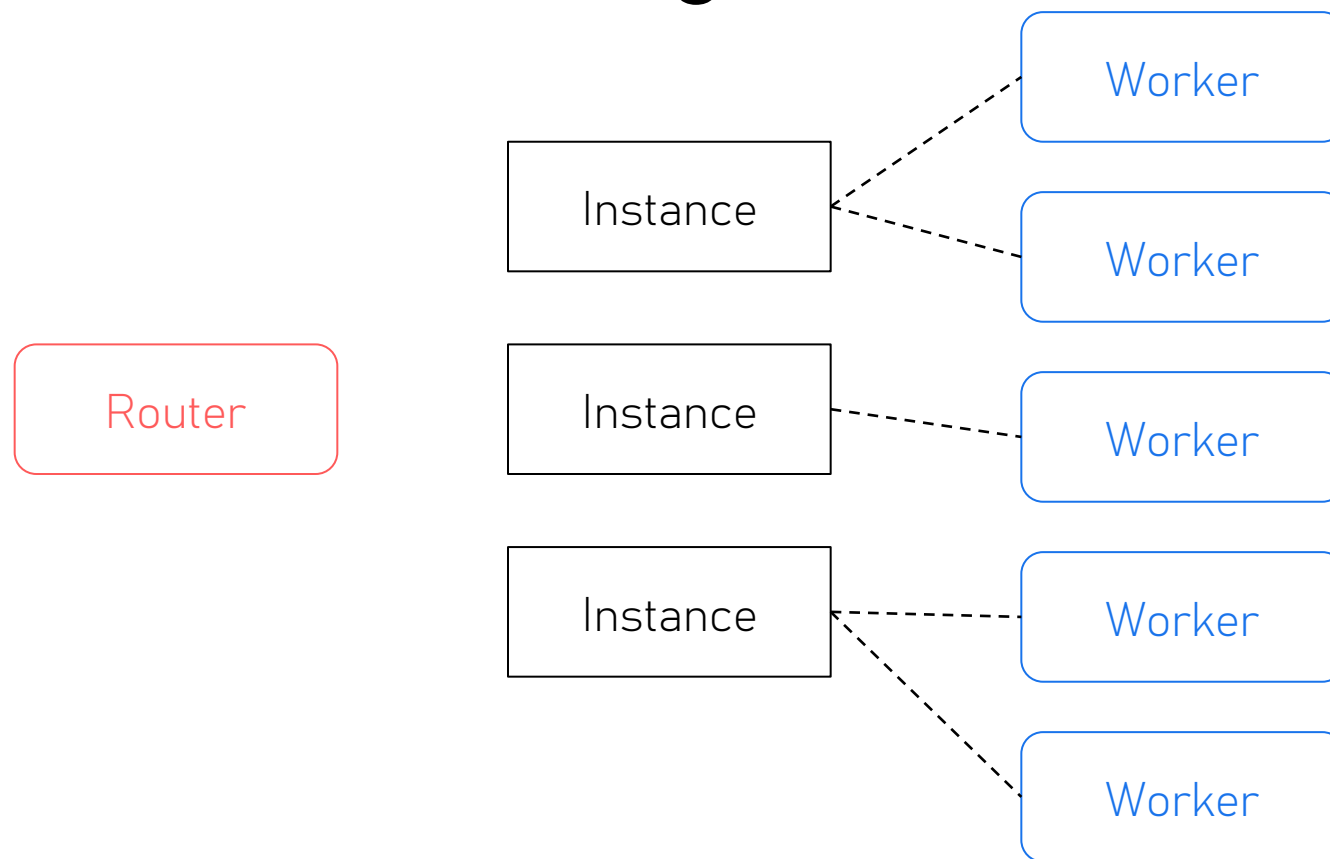

Memory Spike

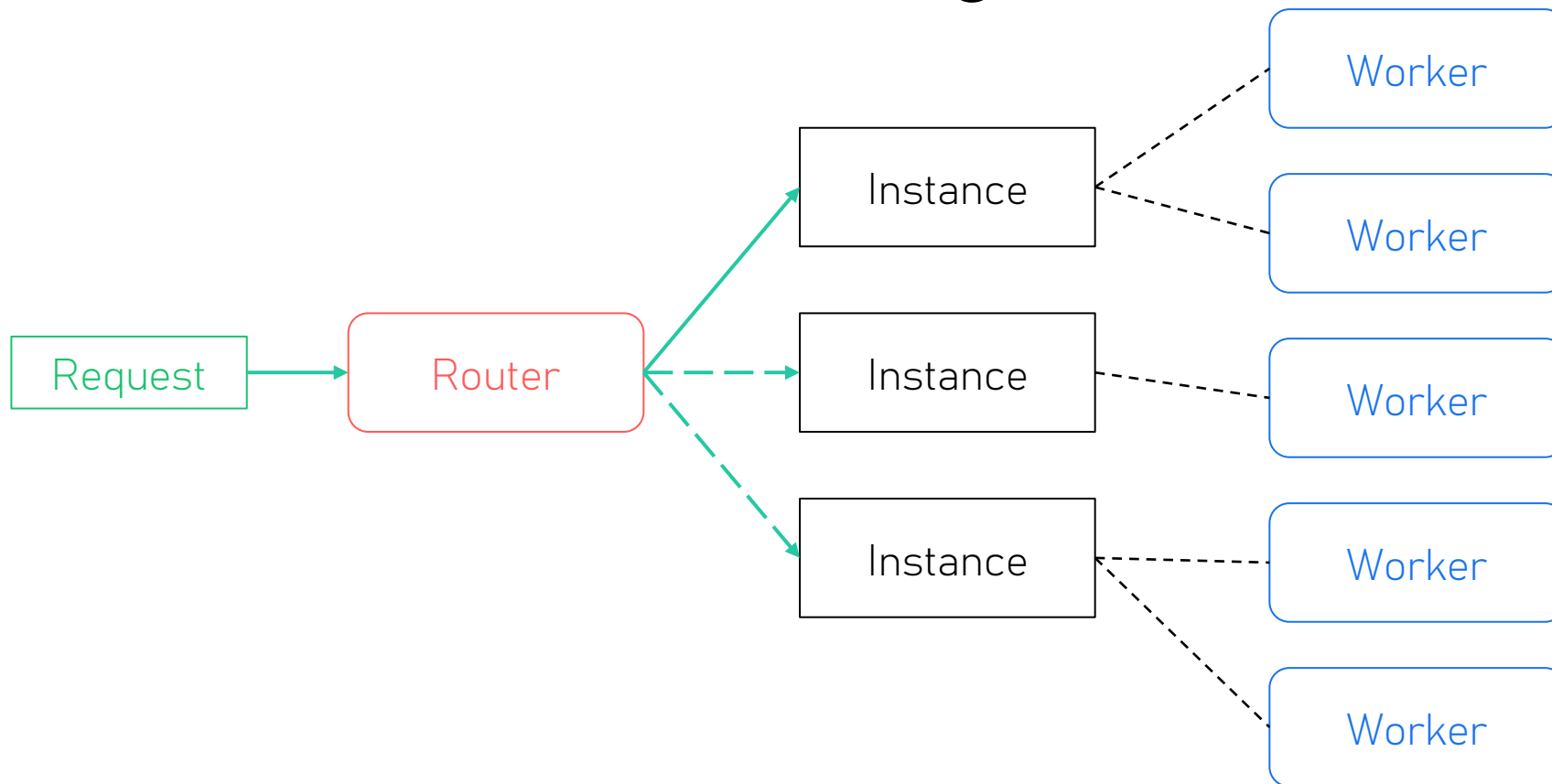
INTRODUCTION TO LLM
INFERENCE SERVING SYSTEMS
CHUHONG YUAN



Multi-Instance Serving

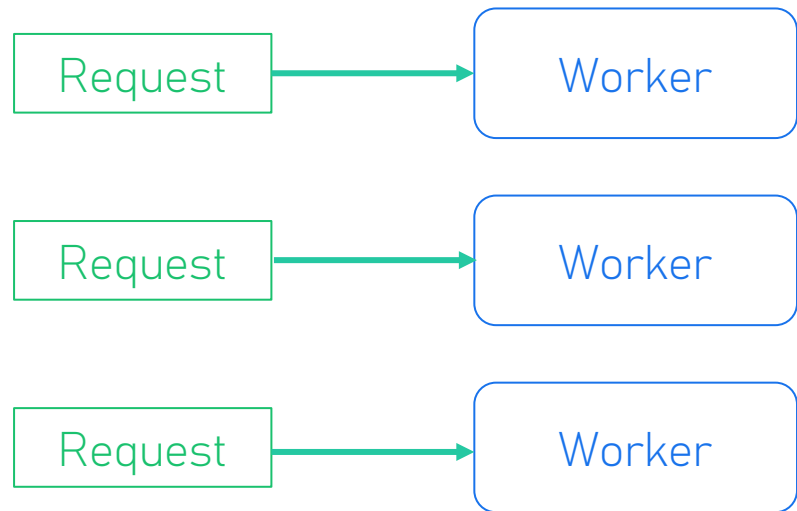


Multi-Instance Serving



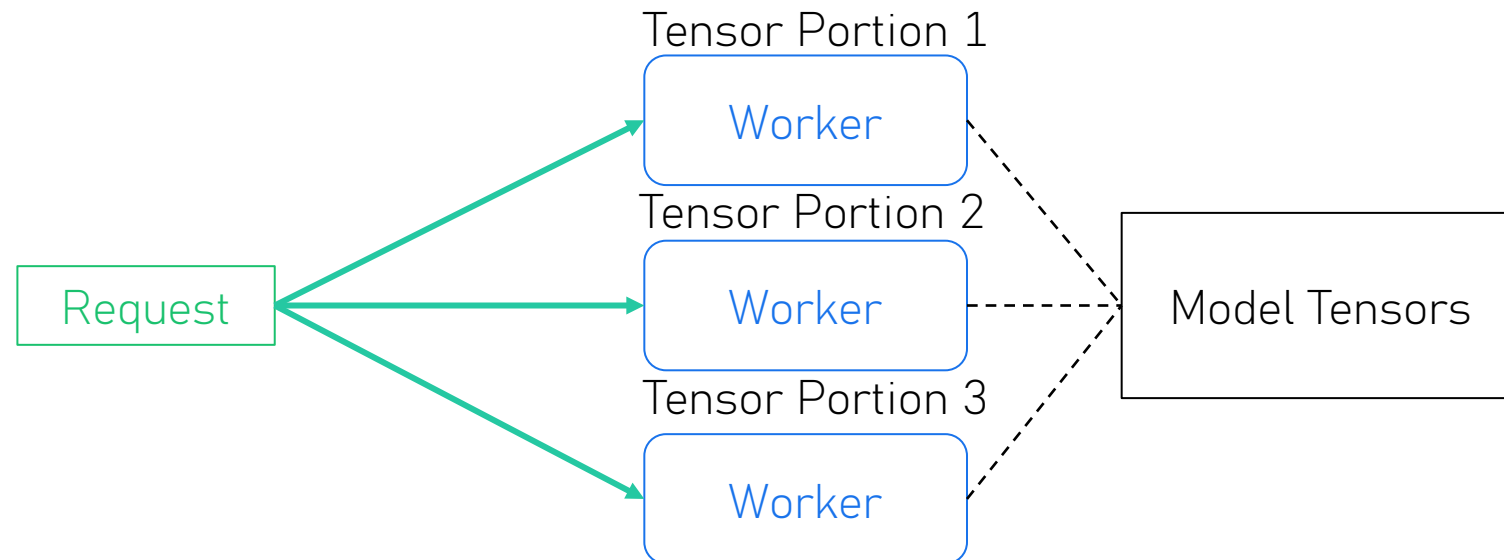
Parallelism Review

- Data parallelism



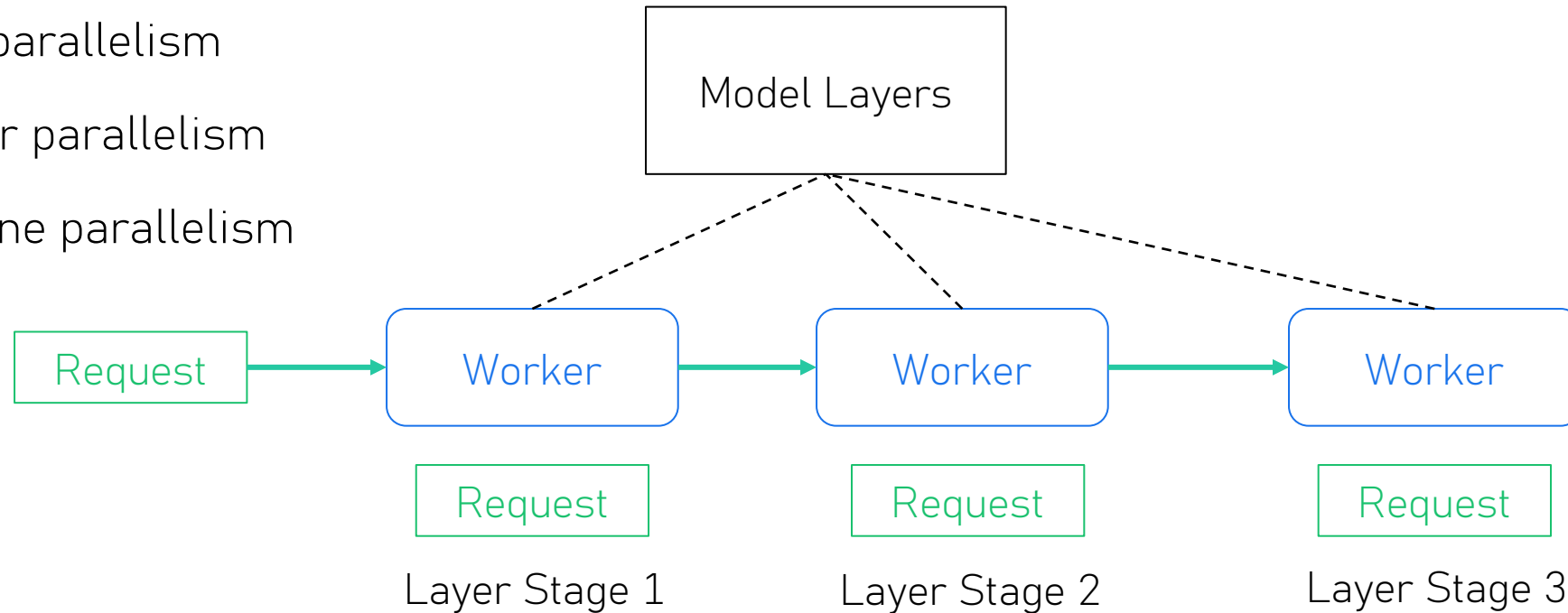
Parallelism Review

- Data parallelism
- Tensor parallelism



Parallelism Review

- Data parallelism
- Tensor parallelism
- Pipeline parallelism



Dynamic Traffic

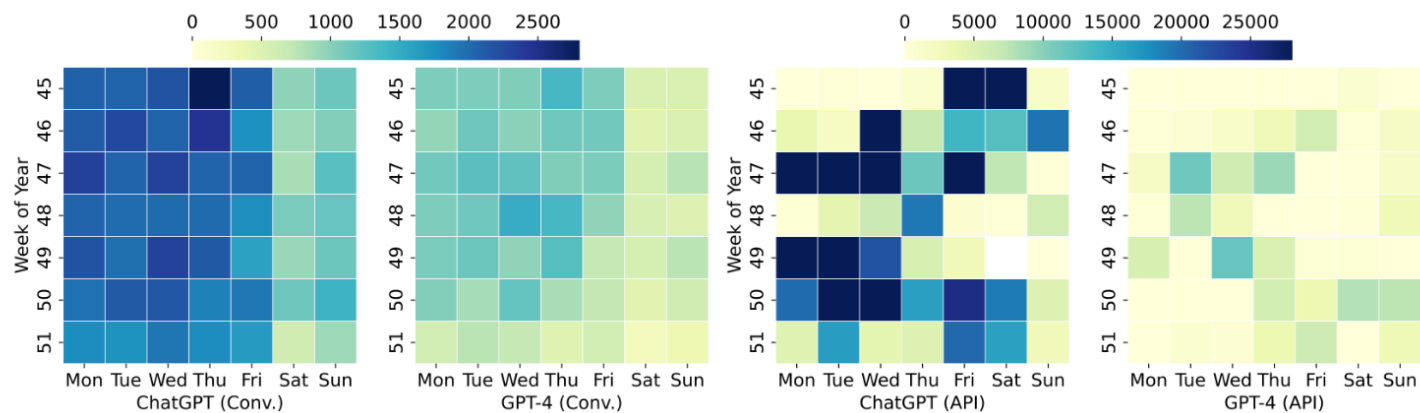


Figure 1: Weekly Periodicity in BurstGPT.

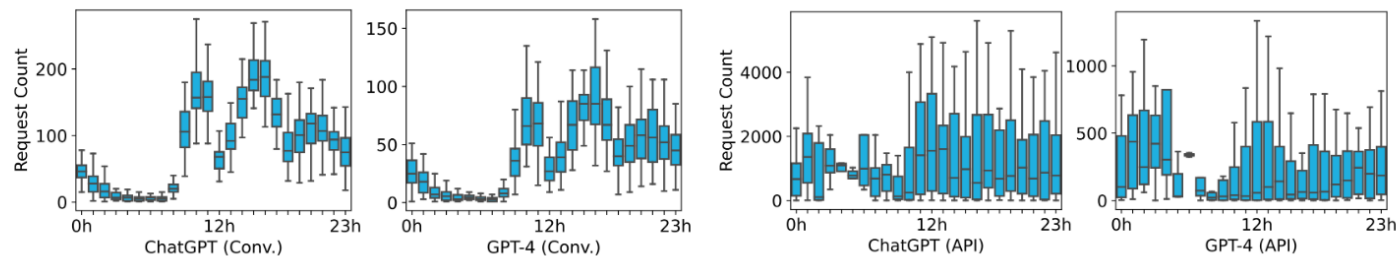


Figure 2: Daily Periodicity in BurstGPT.



Memory Spike

- Memory usage suddenly grows – mainly caused by KV Cache growth
- Factors
 - Dynamic numbers of requests
 - Unpredictable output lengths of requests
- Effects
 - Cap the maximum concurrent requests – throughput
 - Hurt other requests on the same machine – latency

Llumnix



Dynamic Memory Usage

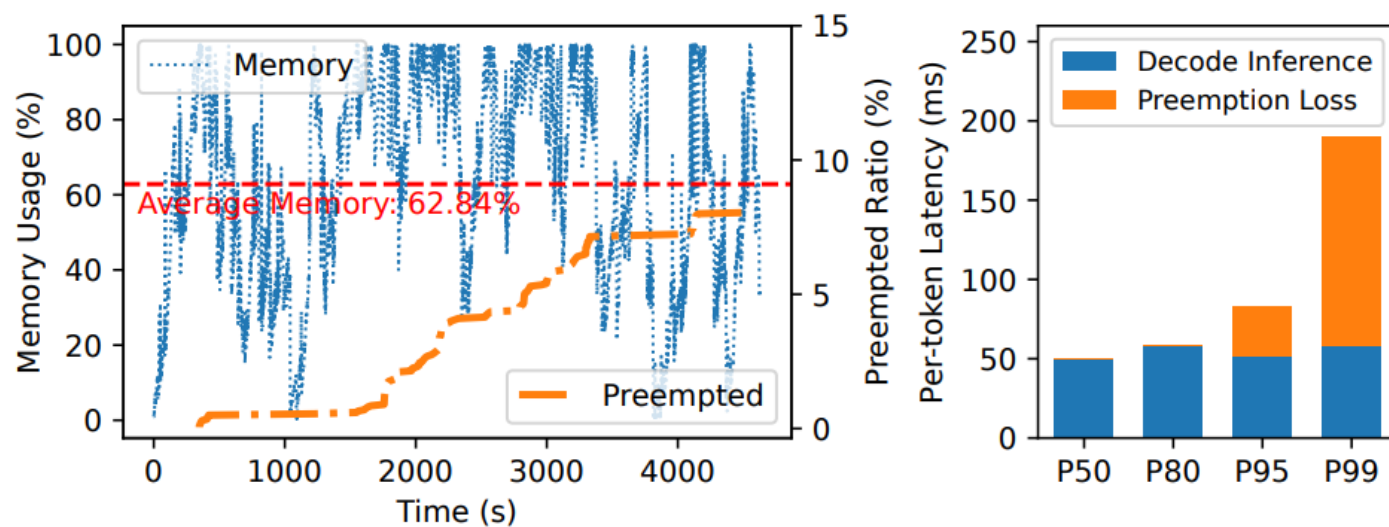


Figure 3: Request preemptions in LLaMA-7B serving.

Preemption

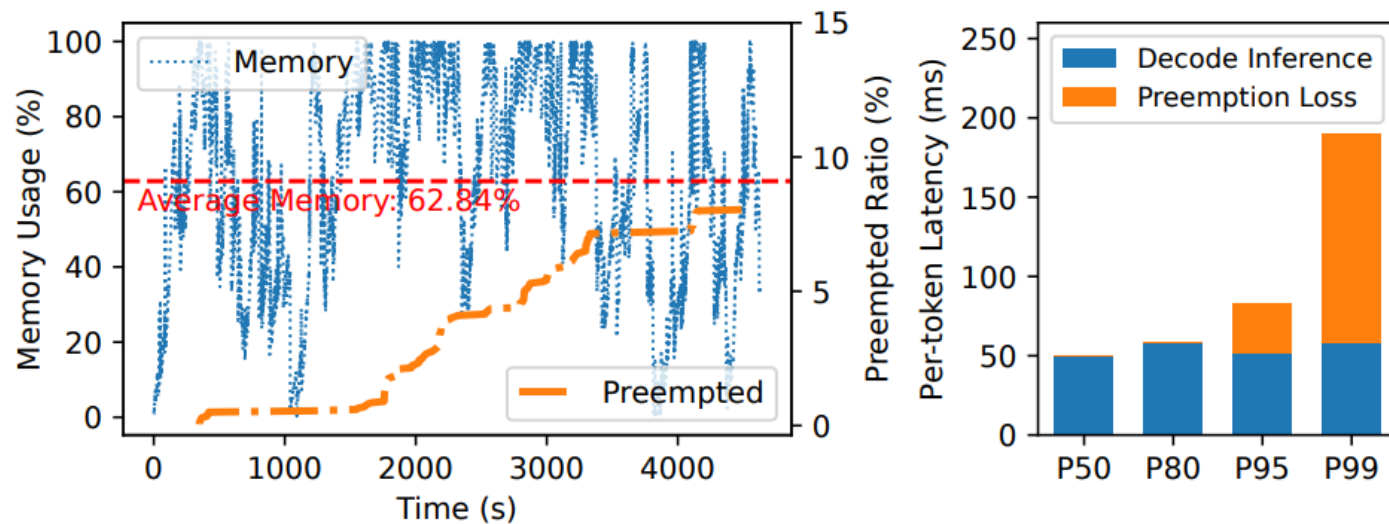


Figure 3: Request preemptions in LLaMA-7B serving.

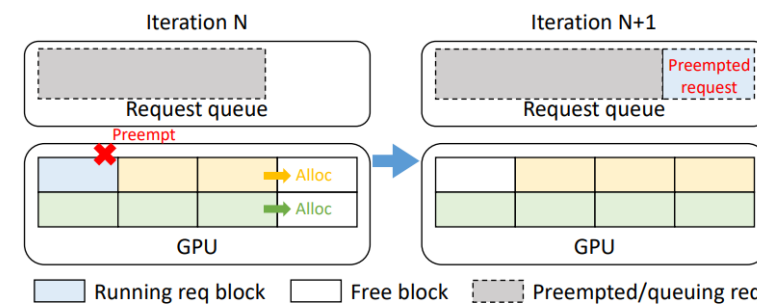


Figure 2: Request queuing and preemption using continuous batching and dynamic memory allocation.

Interference

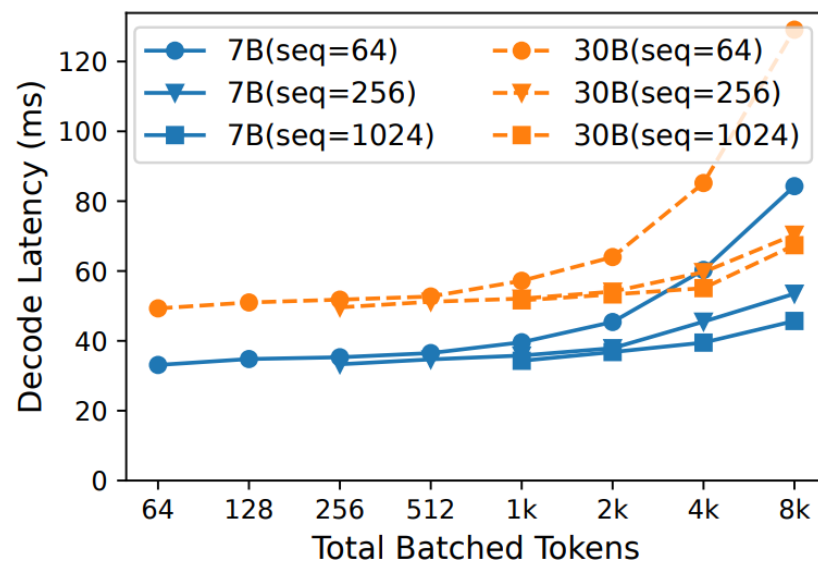


Figure 4: Latencies of one decode step of LLaMA-7B and LLaMA-30B with different sequence lengths and batch sizes.

Memory Fragmentation

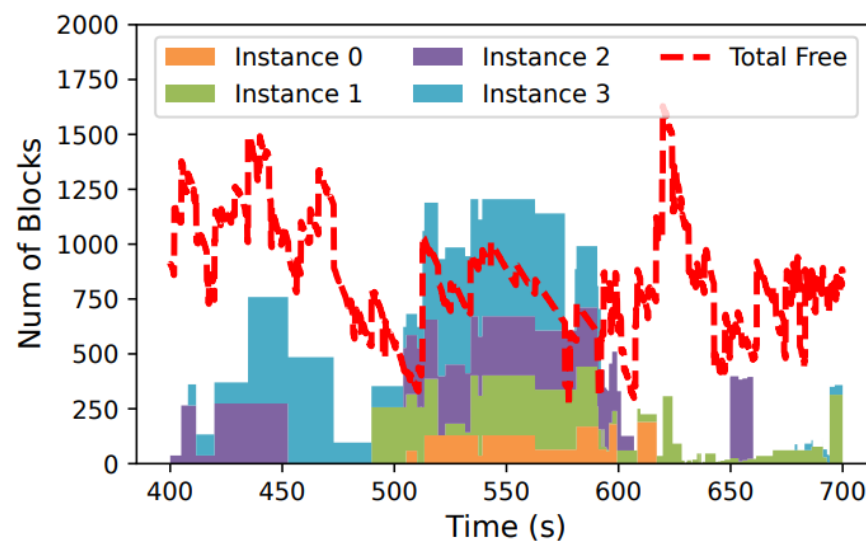


Figure 5: Total free memory vs. demands of the head-of-line queuing requests across four LLaMA-7B instances.

Llumnix Overview

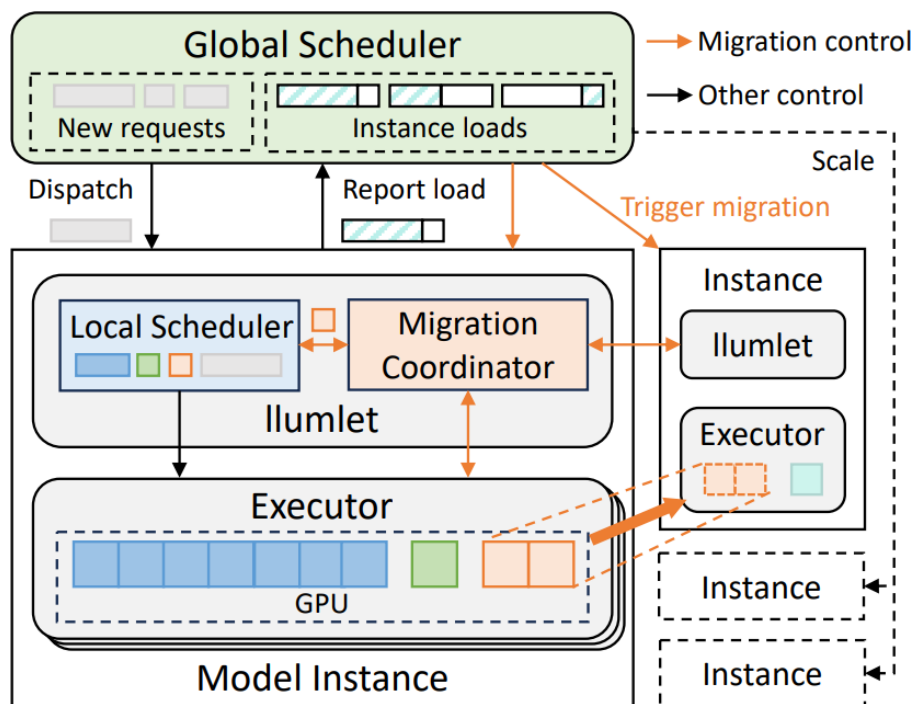


Figure 8: Llumnix architecture.

Applications

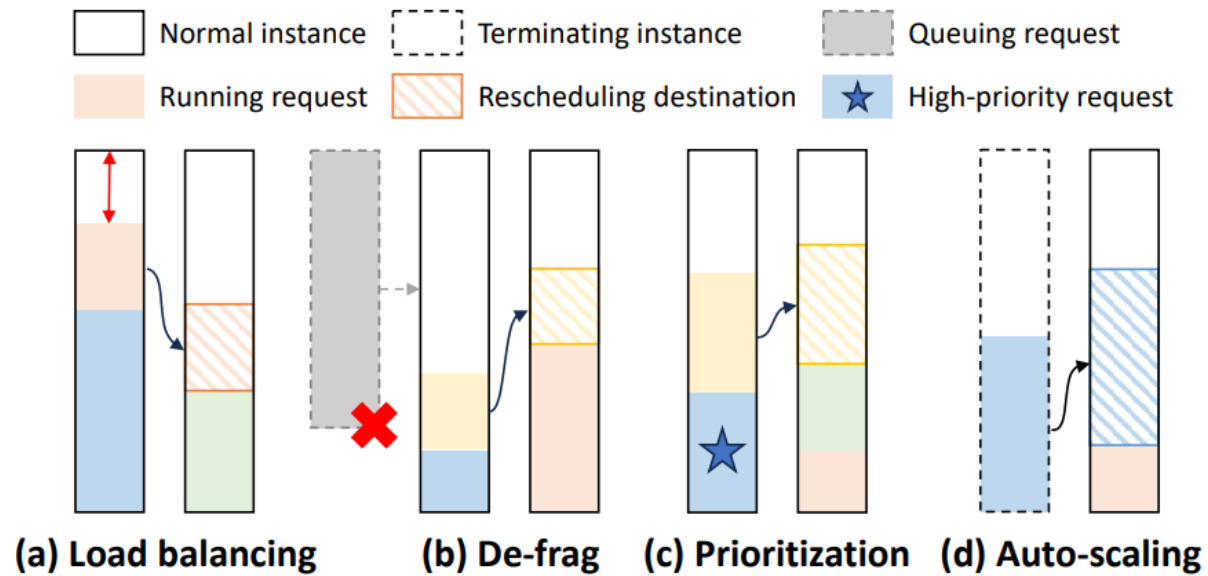


Figure 1: Example rescheduling scenarios in Lumnix.

Requests Live Migration

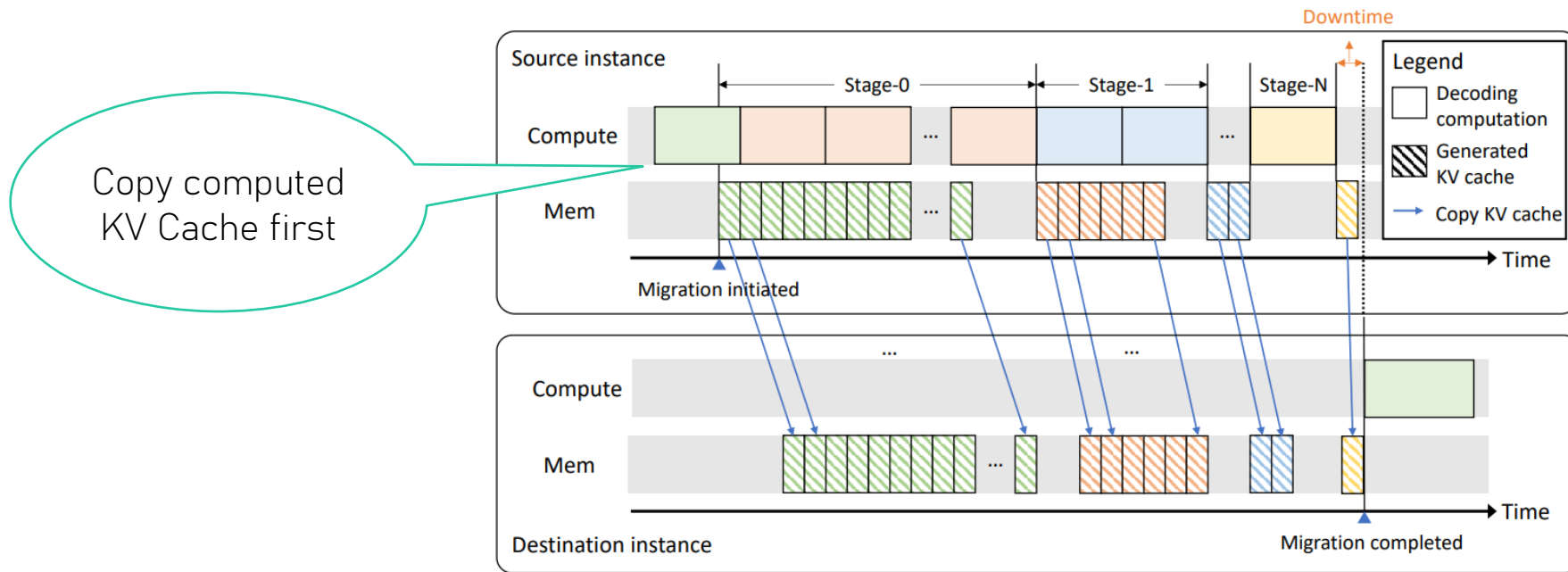


Figure 6: Llumnix adopts multi-stage migration to overlap the computation and KV cache copying for minimal downtime.

Requests Live Migration

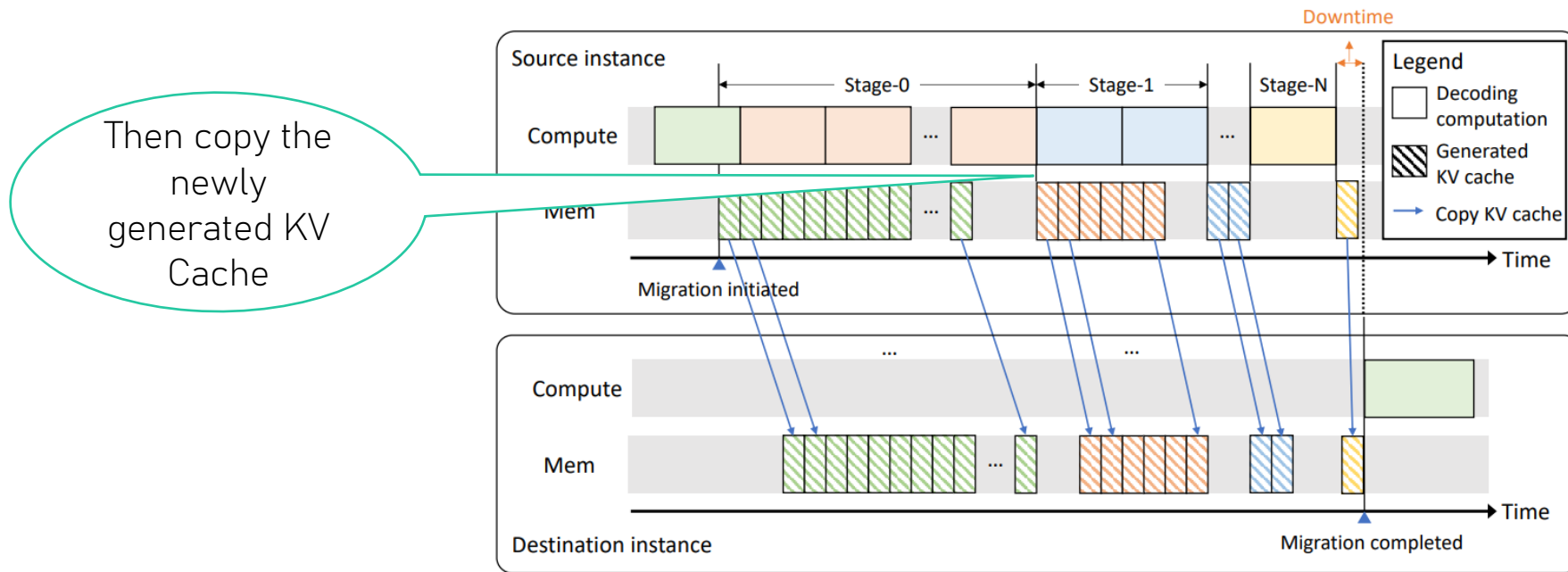
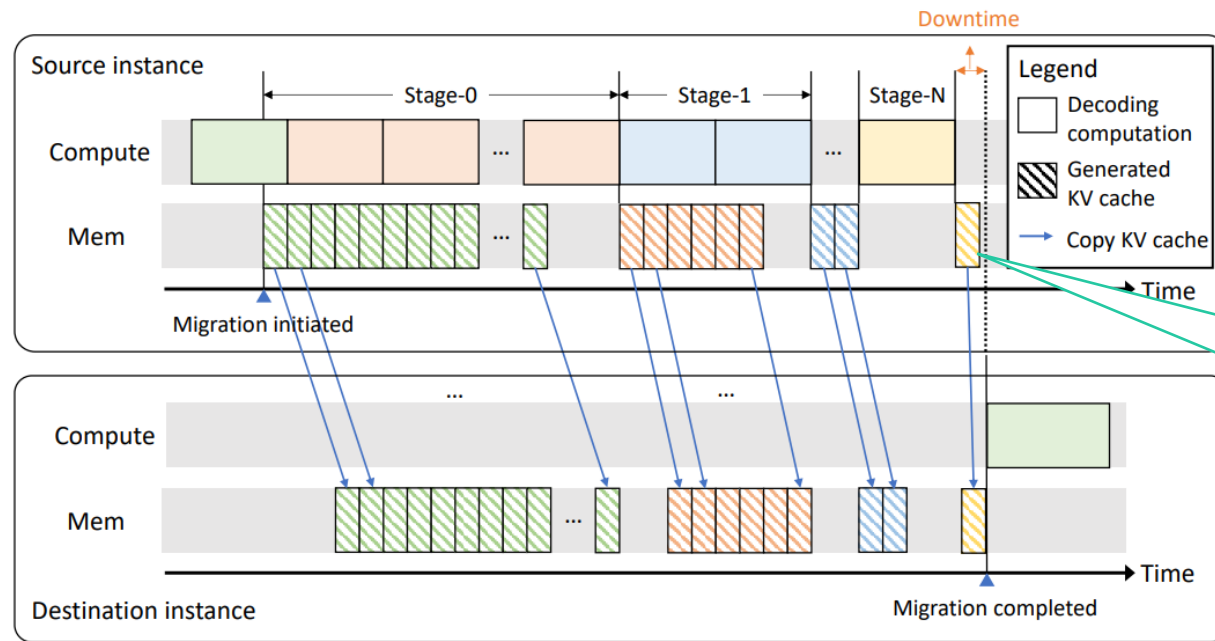


Figure 6: Llumnix adopts multi-stage migration to overlap the computation and KV cache copying for minimal downtime.

Requests Live Migration



The extra cost is from copying the last KV Cache

Figure 6: Llumnix adopts multi-stage migration to overlap the computation and KV cache copying for minimal downtime.

Migration Handshake

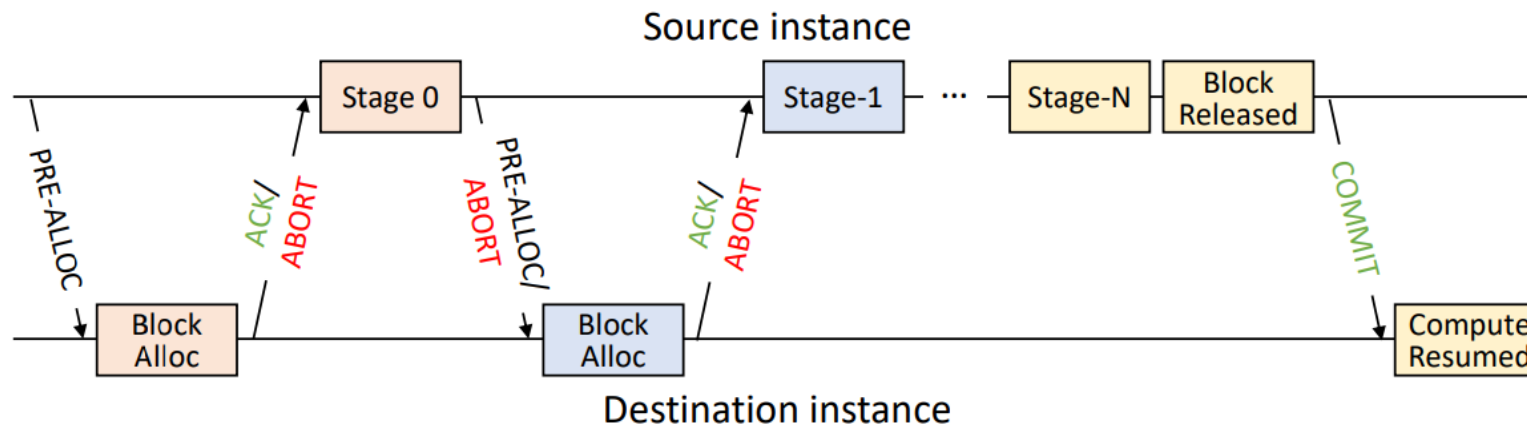


Figure 7: Handshake during migration.

Distributed Scheduling Architecture

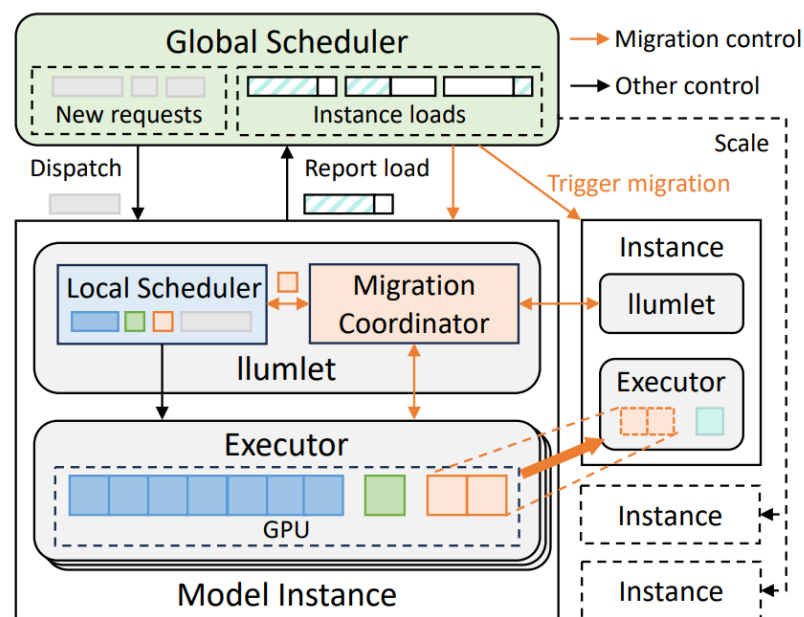


Figure 8: Lumnix architecture.

Distributed Scheduling Architecture

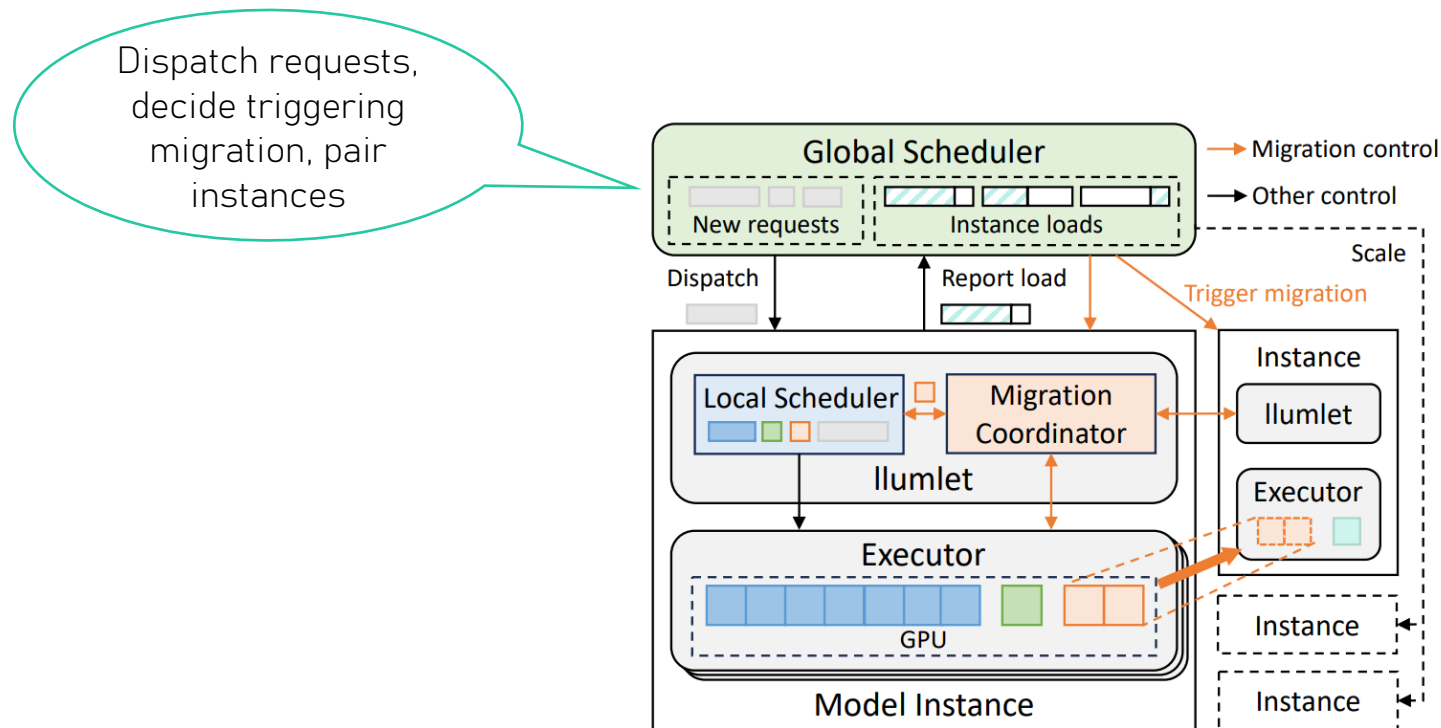


Figure 8: Lumnix architecture.

Distributed Scheduling Architecture

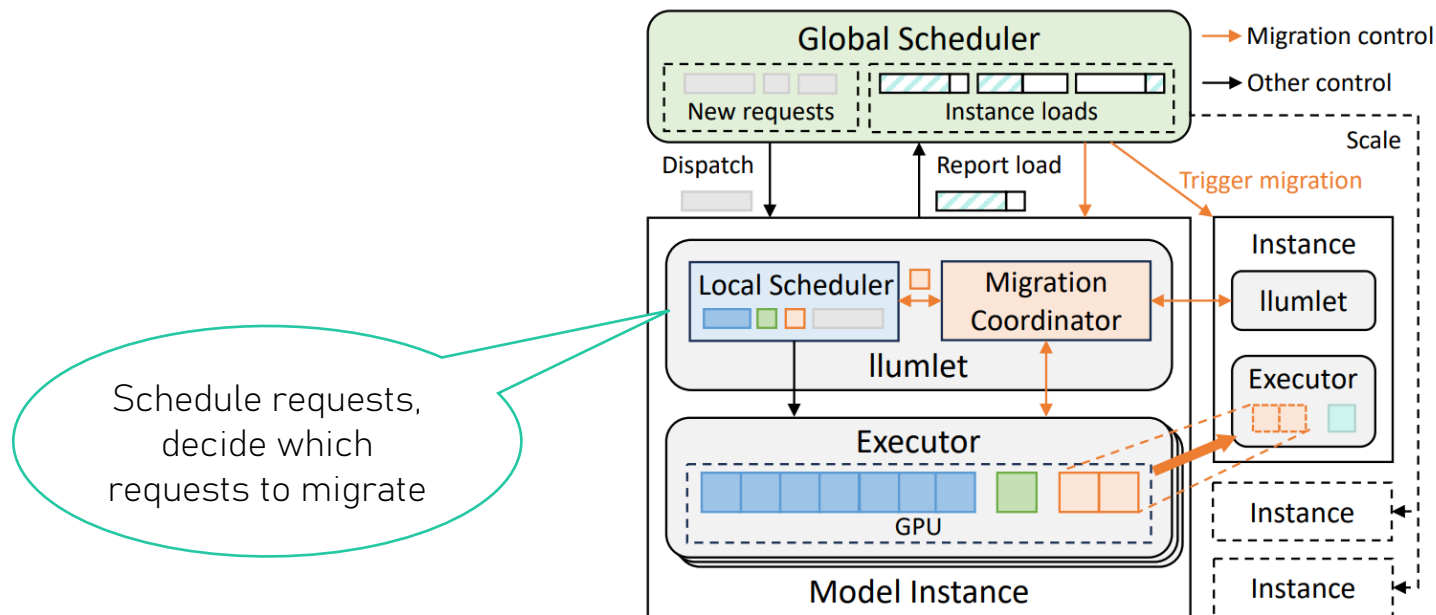
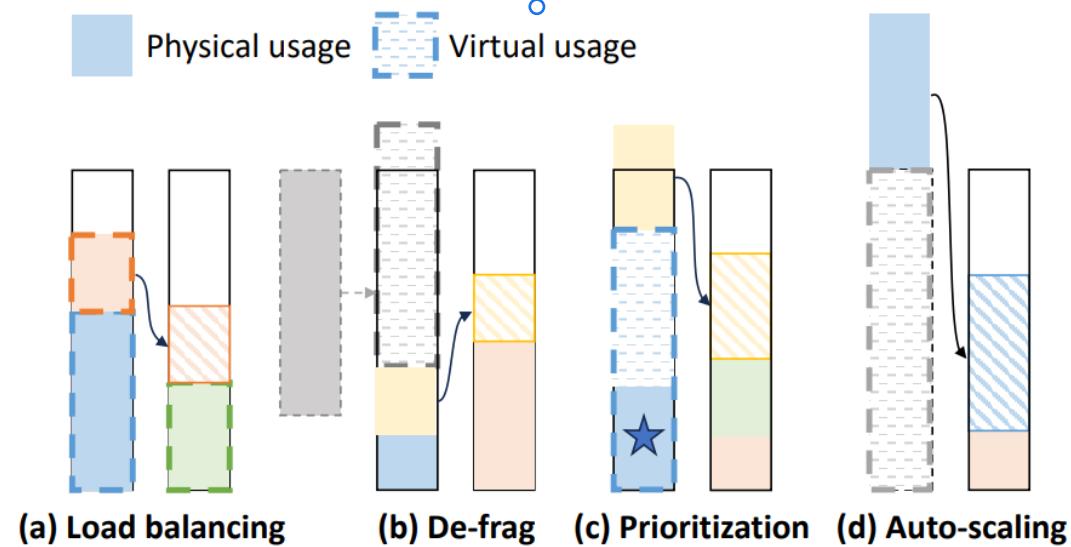


Figure 8: Lumnix architecture.

Dynamic Scheduling Policy



Make all
issues load-
balancing

Figure 9: Llumnix combines virtual usages with a load-balancing policy to unify multiple scheduling goals.



Evaluation – Settings

- GPU: A10 24GB, four in each node, four nodes
- Model: Llama-7B 30B
- Workload: ShareGPT, BurstGPT, self-picked combinations, Poisson and Gamma distributions
- Baselines: round-robin, InFaaS++, Llumnix-base (no priority)

Evaluation – Migration Efficiency

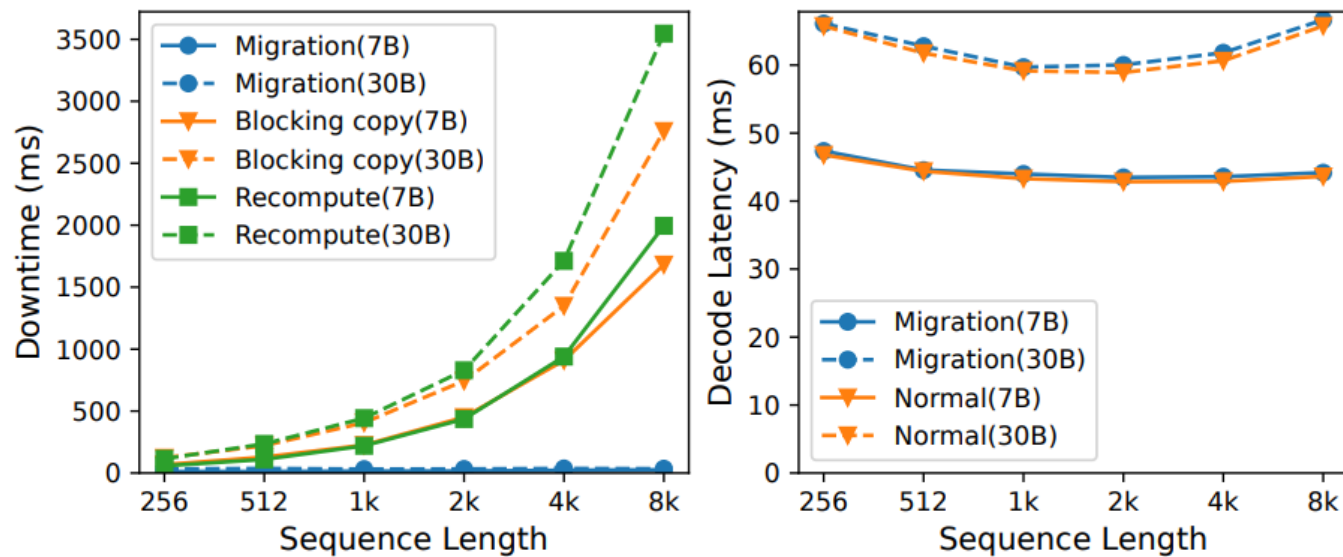


Figure 10: Downtime and overhead of migration.

Evaluation – Serving Performance

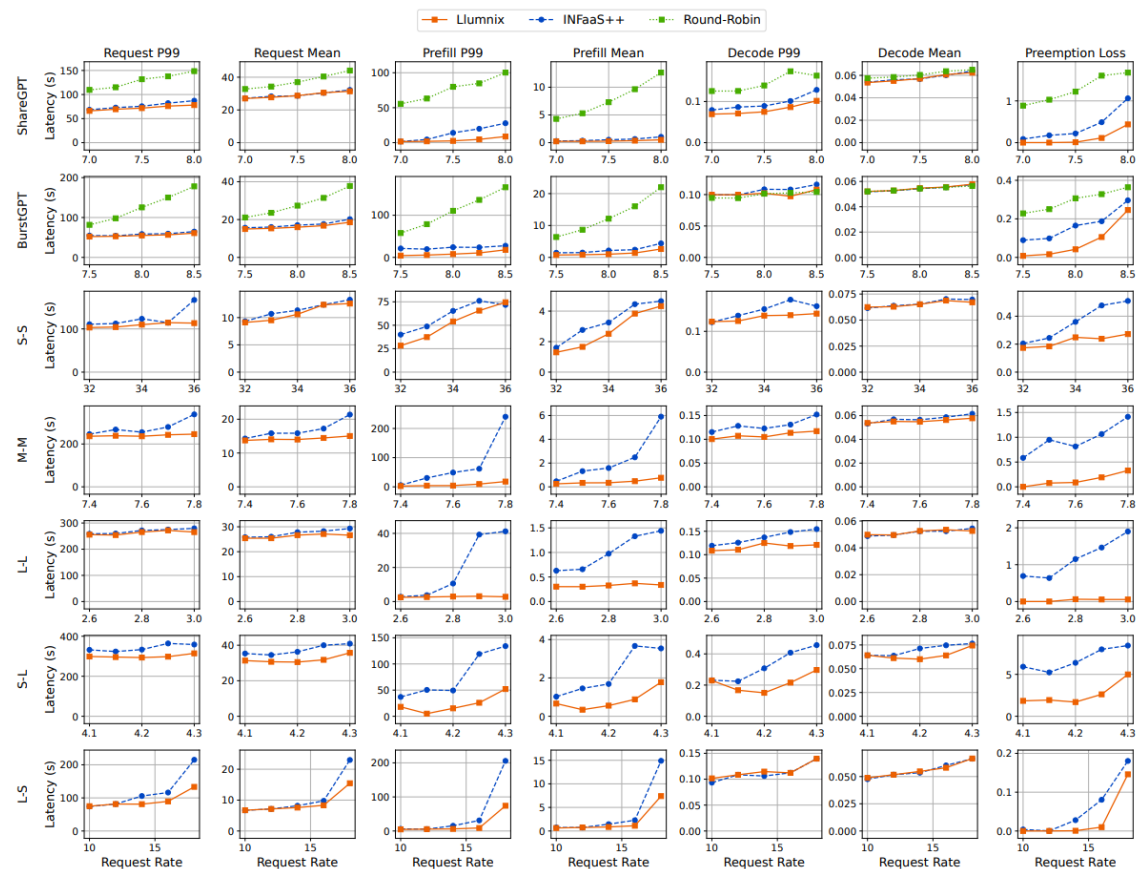
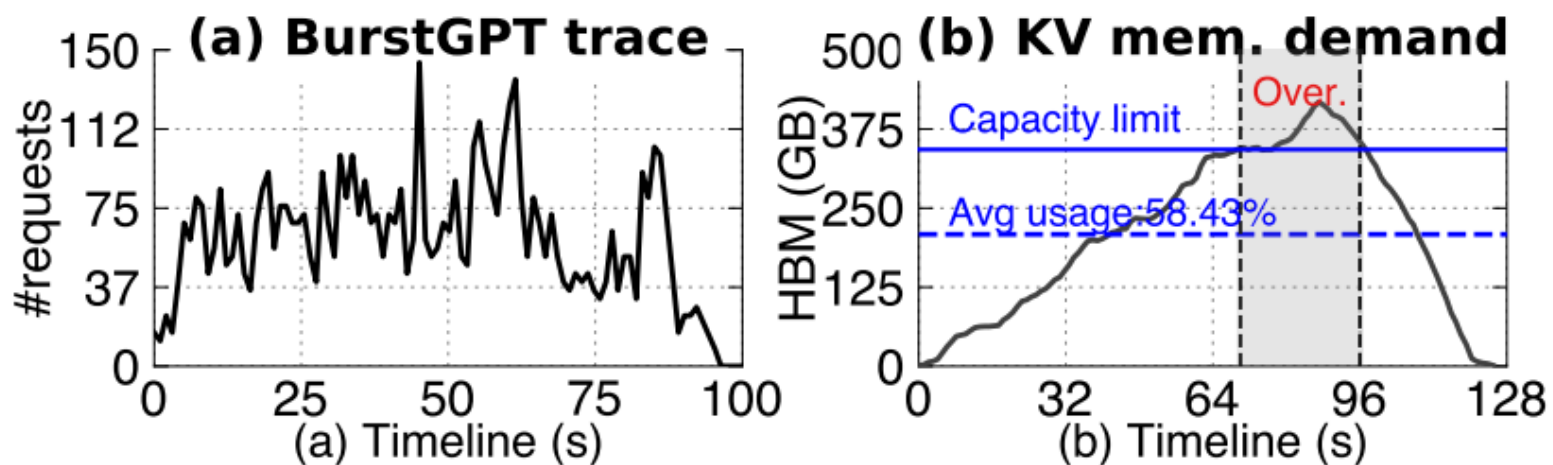


Figure 11: Request end-to-end, prefill, and decode latencies and preemption loss of serving 16 LLaMA-7B instances. Each row shows a set of experiments using a trace with a specific sequence length distribution, as annotated on the Y-axis labels.

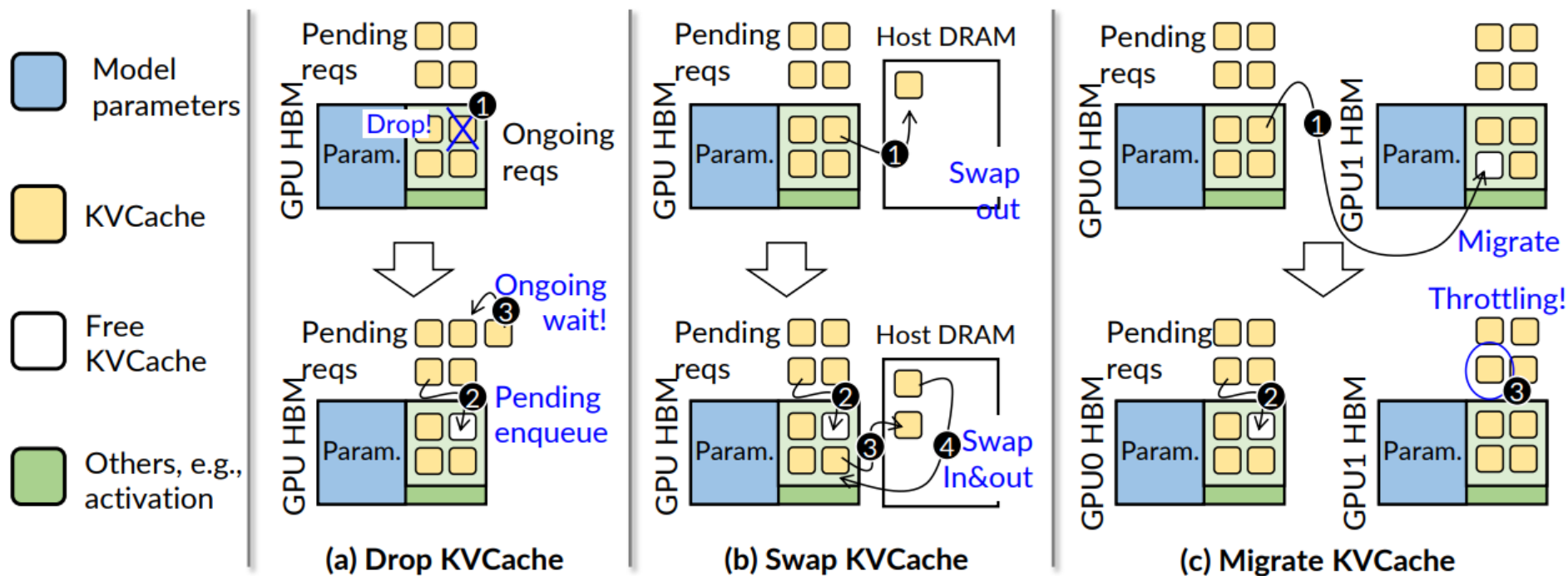
KunServe



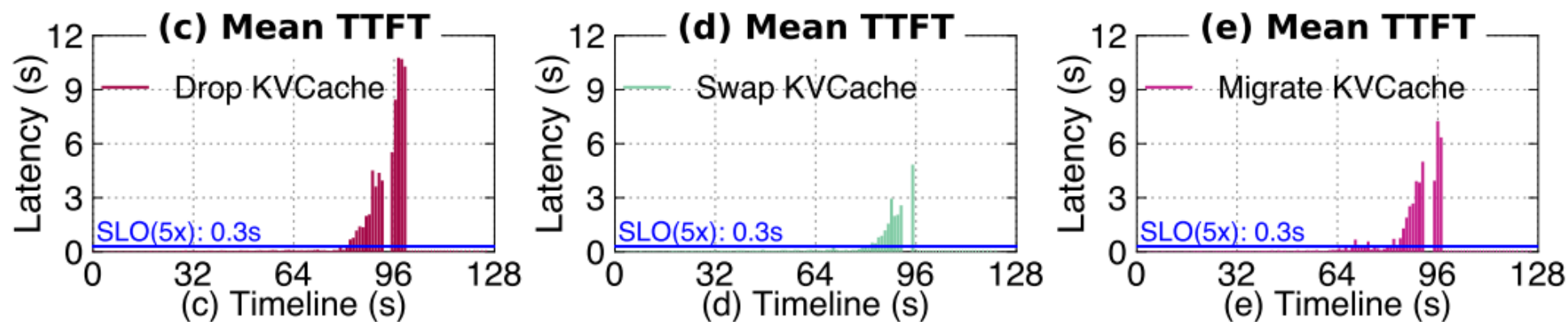
Memory Usage Spikes



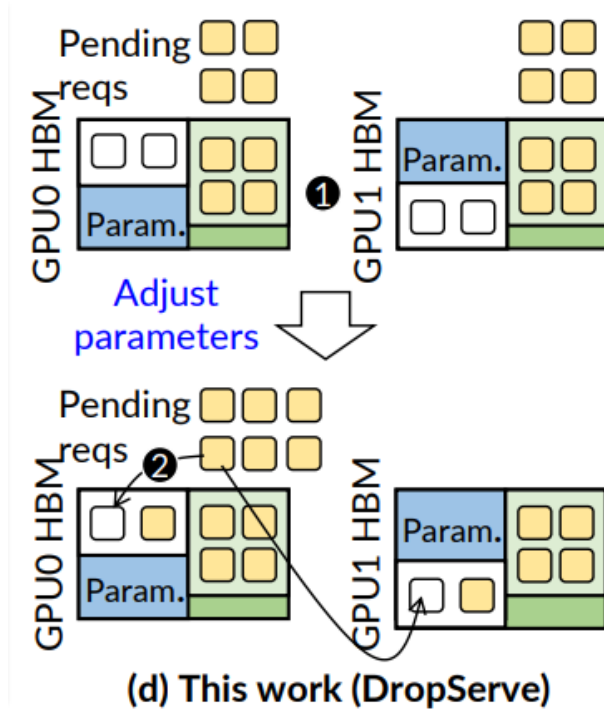
Existing Methods



Existing Methods – TTFT Spikes



Idea – Pipeline Parallelism



- Network requirement usually satisfied
- Easy to set up

Pipeline Parallelism Effect

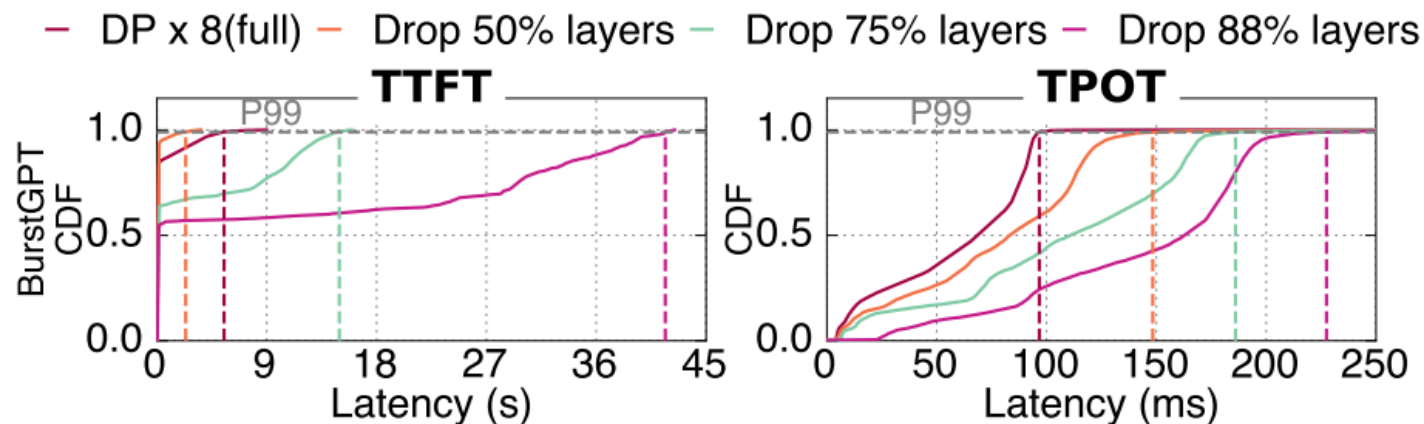


Figure 5: A comparison of the latency of different parallelism on BurstGPT dataset. All setups are evaluated with 8 GPUs.

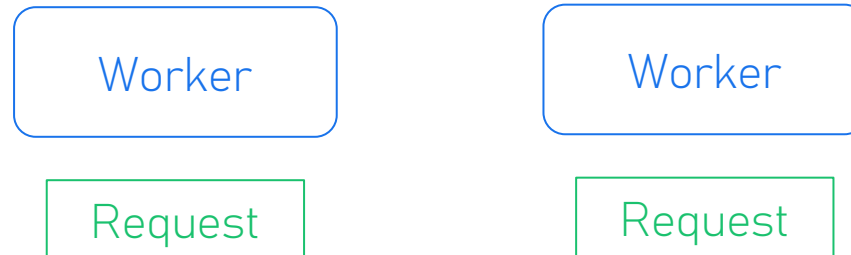


Parameter Drop

- Involve instances as few as possible – greedy method
- Organize all groups of instances in a min-heap (each group starts with 1 instance)
- Pop two smallest groups and drop the duplicated layers in their models
- Repeat the procedure until the freed memory is sufficient

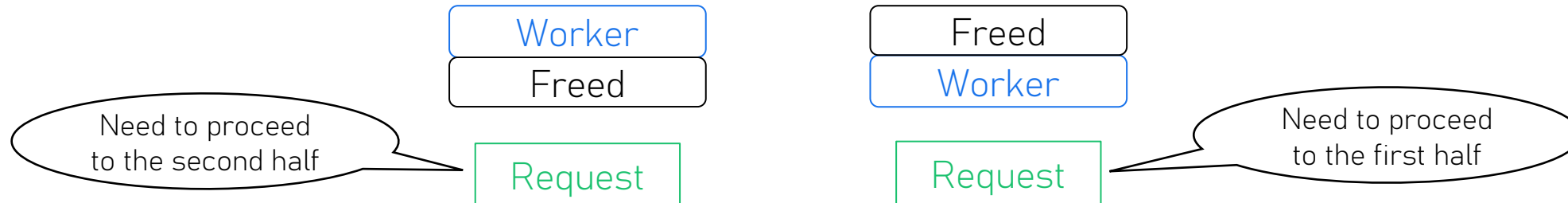
Parameter Drop

- Involve instances as few as possible – greedy method
- Organize all groups of instances in a min-heap (each group starts with 1 instance)
- Pop two smallest groups and drop the duplicated layers in their models
- Repeat the procedure until the freed memory is sufficient
- Need to exchange KV Cache after switching to PP



Parameter Drop

- Involve instances as few as possible – greedy method
- Organize all groups of instances in a min-heap (each group starts with 1 instance)
- Pop two smallest groups and drop the duplicated layers in their models
- Repeat the procedure until the freed memory is sufficient
- Need to exchange KV Cache after switching to PP



Batch Formulation – Bubbles

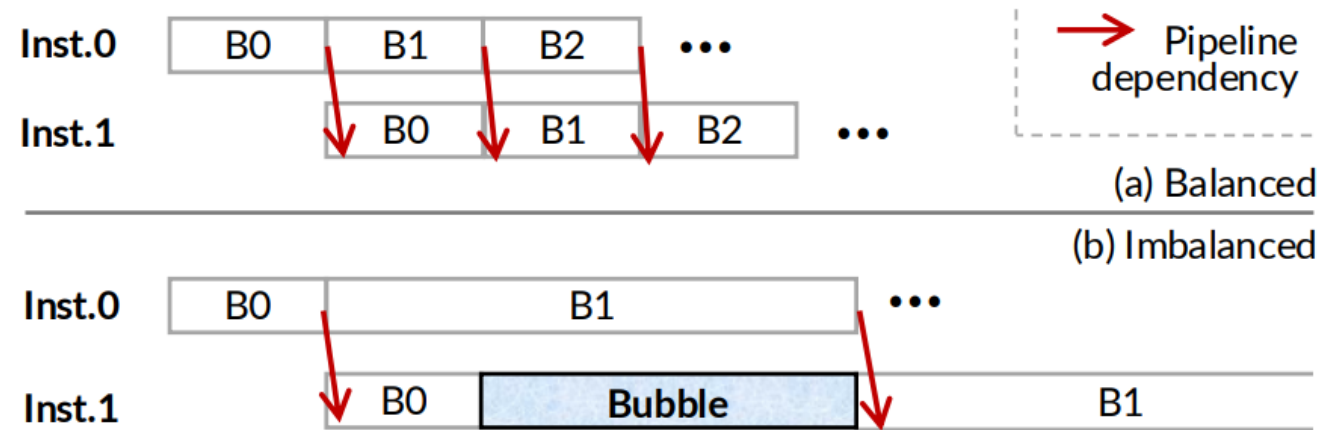
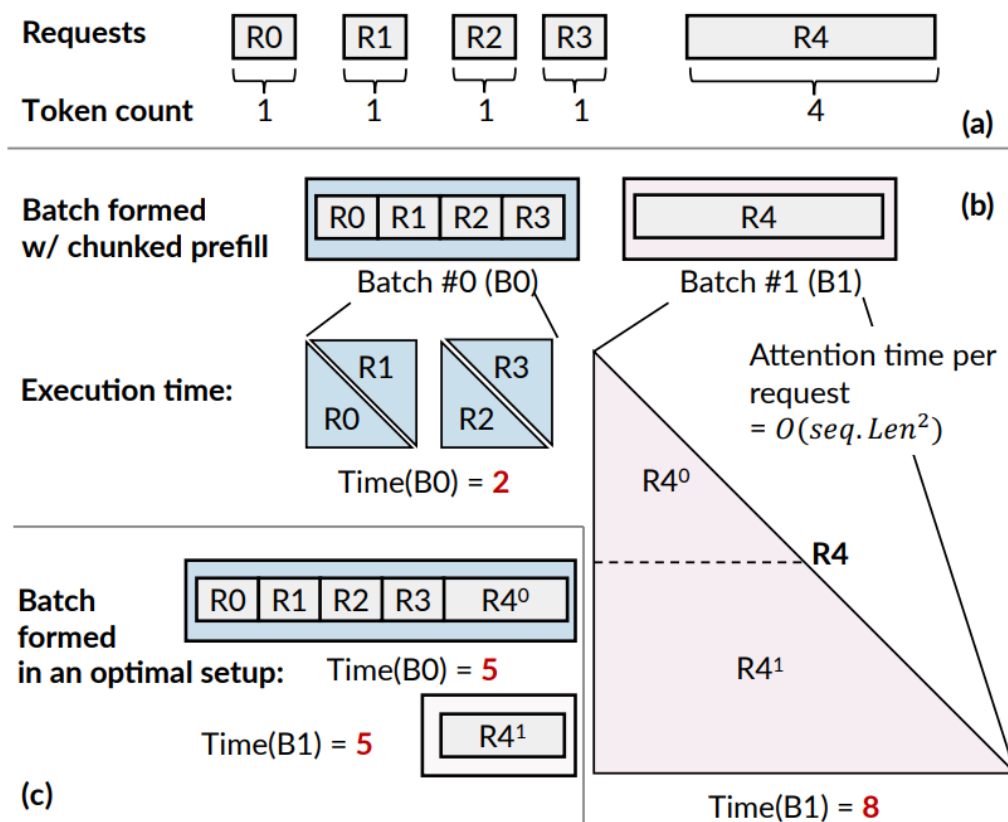


Figure 8: An illustration of pipeline execution bubbles caused by imbalanced execution time of microbatches.

Batch Formulation



Batch Formulation

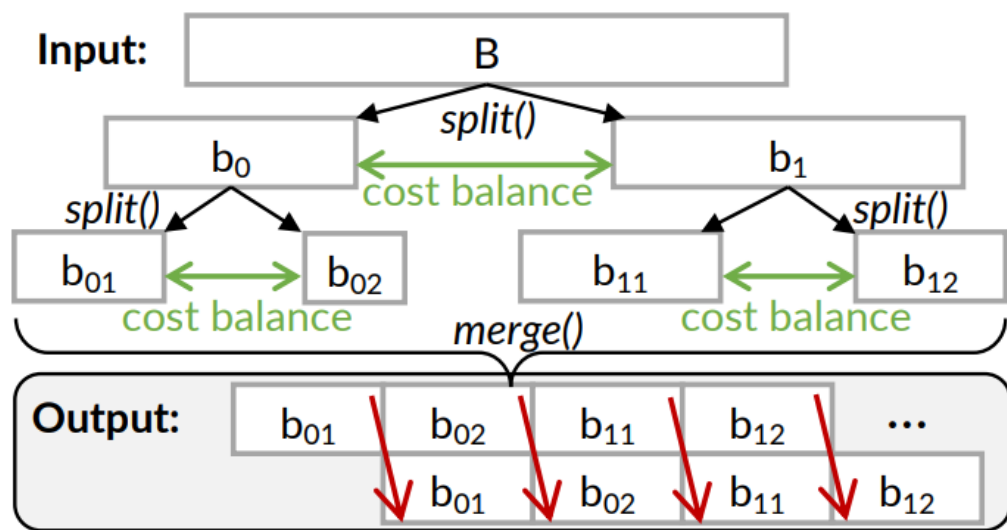


Figure 10: An illustration of how lookahead batch formulation recursively generate balanced microbatches.

$$\text{cost}_{c_{ij}} = \alpha \left(\underbrace{p_{ij} c_{ij}}_{\text{prefix-attn}} + \underbrace{\frac{c_{ij}^2 + c_{ij}}{2}}_{\text{self-attn}} \right) + \beta \underbrace{c_{ij}}_{\text{FFN}} + \underbrace{\gamma}_{\text{other}} \quad (1)$$



Dynamic Restore

- PP still has the bubble issues if there are not enough requests
- When the memory spike passes, pull parameters from other instances



Evaluation – Settings

- GPU: A800 $\times$ 1, H800 $\times$ 2 $\times$ 8
- Model: Qwen-2.5-14B and 72B
- Workload: BurstGPT, ShareGPT, LongBench
- Baselines: vLLM, InterCept, Llumnix

Evaluation – Efficiency

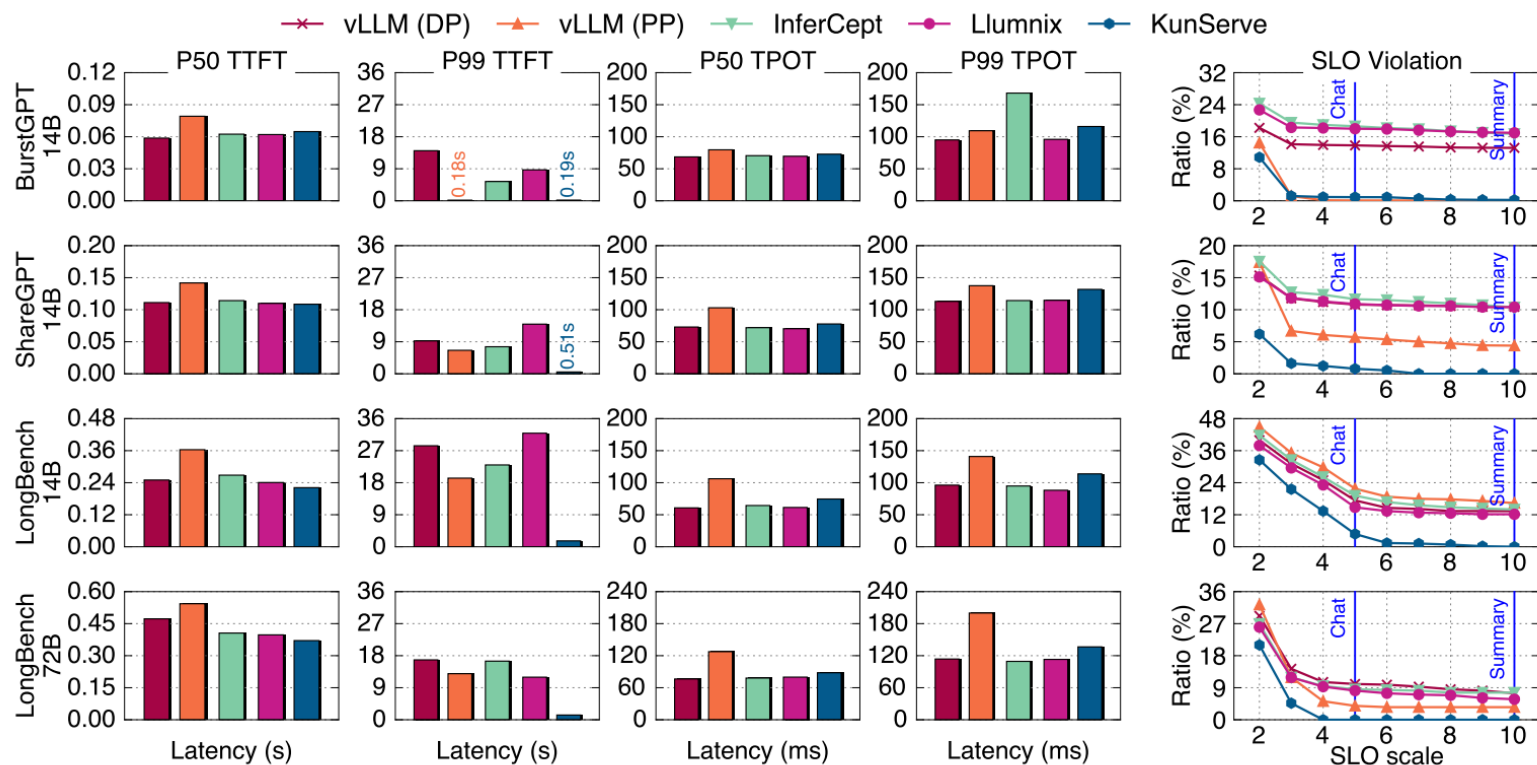


Figure 13: The end-to-end latency results. Column from 1 to 4 is the end-to-end metrics of different workloads. The last column is the SLO violation of TTFT and TPOT with different SLO scales.

Evaluation – Cost Estimation

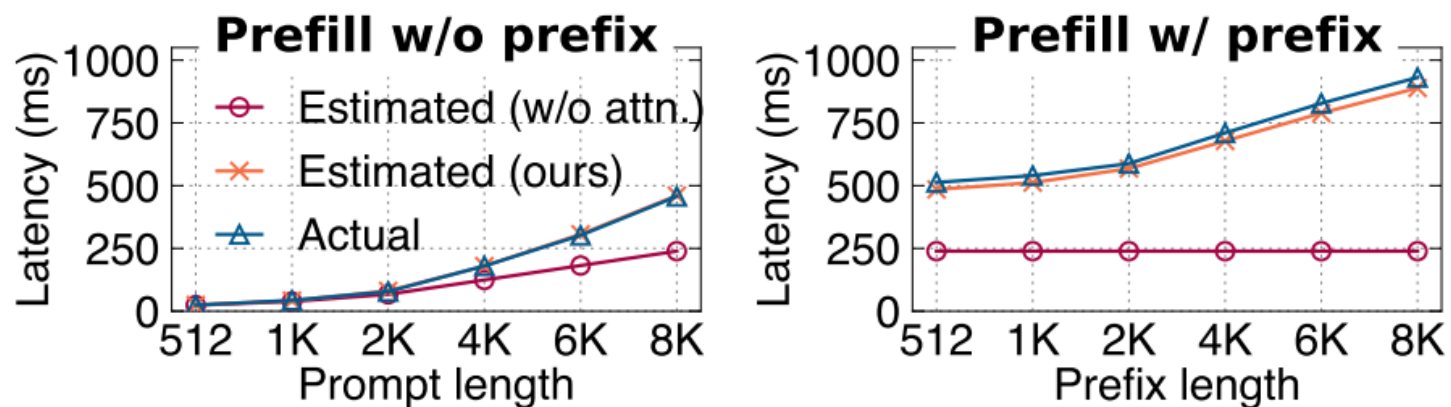


Figure 15: A comparison of execution latency estimated with our cost model and the real execution time of a Qwen-2.5-14B model in A800 GPUs. Left: the execution without prefix attention while right: the execution with prefix attention.

Q & A

